

Boxed: A Docker-Based Code-Execution Substrate for Autonomous Code-Generating Agents

Akshay Kumar
Independent Researcher
United States
akumar8@mt.iitr.ac.in

Abstract—Autonomous large language model (LLM) agents write, execute, and iterate on code inside their reasoning loop, so the sandbox sits on the hot path of an agent task. Operators choose among roll-your-own Docker, which is fast to prototype but is routinely deployed without a read-only root, capability drops, process caps, or egress control; hosted sandbox services, which are hardened but move the operator’s code and data to a third party; and research virtual machine monitors, which provide strong isolation but target a boot-once serverless model. We present Boxed, an open-source substrate for agentic code execution built from a self-hosted Go control plane, a 1.32 MiB Rust in-sandbox agent that streams stdout, stderr, and on-disk artifacts over JSON-RPC 2.0, and a driver interface whose four-method lifecycle core is intended to admit backends other than the Docker driver that ships today. We measure the Docker driver against the Docker Engine API called directly on the same host, image, and command: across 5 runs of 200 lifecycles, Boxed’s median create+exec+destroy latency is 178 ms against 153 ms for raw Docker with the same hardened configuration, a control-plane and agent cost of 25 ms (bootstrap 95% CI 23–27 ms). Applying the hardening to raw Docker is 63 ms faster than stock defaults. A twelve-vector escape probe with post-condition checks read from the host denies 12 of 12 vectors on the hardened driver; the same run scored with the earlier signature-matching classifier disagrees on 2 vectors, which quantifies how much that method misreports. Boxed is MIT-licensed with the harness and raw traces.

Index Terms—sandboxing, code execution isolation, large language models, autonomous agents, containers, Docker, JSON-RPC

I. Introduction

The 2024–2026 wave of autonomous code-generating agents, from Yang et al.’s SWE-agent [1] and AutoGPT-style harnesses through commercial products such as Cursor, Devin, and Claude Code, has changed what “running untrusted code” means in practice. Executable code has been argued to be a more effective action space for an agent than text or JSON [2], and an agent on that pattern emits dozens of short-lived programs per task: it generates a snippet, executes it, inspects stdout, reads back a generated file, and iterates. The sandbox is not a verification step at the end of the pipeline; it is a component of the agent’s decision loop, and its latency is paid on every iteration.

Today’s options for hosting that loop fall into three groups, and each leaves an operator who cares about cost,

latency, and control over deployment without a complete answer.

Roll-your-own Docker is the common choice. It is fast to prototype but is routinely shipped without seccomp profiles, network egress filters, read-only root filesystems, or process and memory caps, leaving the host one `rm -rf /` away from compromise [3]. Measurement studies of deployed containers report the same pattern at scale [4], and cgroup-based resource isolation has itself been shown to be evadable [5].

Hosted sandbox services (E2B [6], Modal Sandbox [7], Cloudflare Sandbox [8]) ship hardened isolation but require the operator to transmit every byte of generated code, execution output, and intermediate artifact to a third party, which conflicts with some operators’ deployment and data-handling requirements.

Research VMMs such as Agache et al.’s Firecracker [9] provide the right primitives but target AWS Lambda’s boot-once, amortized invocation model and ship without an in-guest agent that can stream artifacts back to the caller.

Agentic workloads call for a sandbox substrate that treats the LLM client as a first-class citizen: an isolation backend chosen at deployment time, an in-sandbox agent that streams standard output and on-disk artifacts as they appear, and a self-hosted deployment in which the operator controls the control plane and its workload data.

A. Problem Statement

An autonomous coding agent executes untrusted, machine-generated code many times within a single task. Three properties are required at once. The isolation boundary must be strong enough to contain code the operator never reviewed. The per-execution latency must be low enough that the sandbox does not dominate the agent’s loop. The deployment must leave the operator in control of the keys and the data, because agent workloads routinely carry proprietary source and customer records into the sandbox.

No existing substrate satisfies all three. Hosted sandbox services meet the first two but place a third party inside the trust boundary and expose no control over the isolation backend. Self-hosted container runtimes leave the operator sovereign but couple the control plane to

one isolation technology, so strengthening the boundary later means rewriting the orchestration layer. The gap addressed here is a substrate that keeps the control plane, the keys, and the data under the operator’s control while treating the isolation backend as a deployment-time choice.

Two questions follow from that goal and are answered by measurement in Section VI. First, what does the control plane and agent cost over calling Docker directly, on the same host, image, and command? Second, once the Docker driver applies the hardening that roll-your-own deployments omit, which of a standard set of container-escape vectors does it stop, judged by state observed after the attempt rather than by strings in the attempt’s output?

B. Objectives

- Define an isolation interface narrow enough that container runtimes, microVMs, and WebAssembly sandboxes could be interchanged behind it, so that the security boundary can be strengthened without changing the control plane.
- Deliver execution output incrementally, so that an agent can react to partial results before process exit, and return files produced during execution as structured events on the same channel.
- Support self-hosted deployment without requiring a Boxed-operated service to receive workload data.
- Quantify the cost and the limits of the resulting design on commodity hardware against a raw-Docker baseline, and report the isolation failures the configuration does not prevent.

C. Contributions

- A Go driver interface whose four-method lifecycle core (Create, Start, Stop, Connect) is the contract a backend must satisfy, so that the isolation backend is a deployment-time configuration choice. The release implements one backend, Docker; no microVM or Wasm backend exists yet, and the paper claims only what the Docker driver does.
- A Rust in-sandbox agent that streams stdout and stderr chunks as discrete JSON-RPC events instead of buffering until process exit, mirrors files written under a watched directory back to the caller as artifact events, and reports the child’s real exit status.
- A self-hosted control plane with an operator-configured API key. It is a single-instance authentication mechanism, not a credential-management or multi-tenant system.
- A released benchmark harness and campaign: lifecycle latency against a raw-Docker baseline measured with the Engine API on the same host, throughput under concurrency with repeated sweeps and confidence intervals, idle per-sandbox overhead, a twelve-vector escape probe scored by post-condition checks read

from the host, and an end-to-end agent trace on official HumanEval tasks with the model, prompts, token counts, and exit codes recorded. Raw CSV and JSONL traces ship with the source, alongside the earlier pre-hardening traces for audit.

- A negative-results comparison of two ways to score an escape probe. The signature classifier used in the earlier version of this work and the post-condition checks used here are run on the same attempts, and the vectors on which they disagree are reported.

II. Background and Related Work

A. Container-based isolation

Linux containers built on namespaces, cgroups, and seccomp underpin nearly every development sandbox. They are cheap to start, roughly 50–200 ms when images are warm, which is why they dominate workloads that create and destroy execution environments frequently. That speed has a security cost: containers share the host kernel, exposing a substantially larger attack surface than hardware-isolated alternatives. CVE-2019-5736 [3] is the canonical reminder that a single bug in runc can turn a container into a host. Lin et al. [4] ran 88 of 223 collected exploits from inside a default-configured container and found that 50 succeeded, that capabilities, seccomp, and MAC do more to stop privilege escalation than namespaces and cgroups, and that 11 exploits still broke isolation. That ranking is why the hardening in Section V-B starts with the capability drop. Hardened container runtimes such as gVisor [10] interpose a userspace kernel between the workload and the host; Young et al. [11] quantify its syscall-interception cost at 2–5×, and Anjali et al. [12] compare Linux containers, gVisor, and Firecracker on CPU, network, memory, and file system microbenchmarks and trace the kernel code each executes, finding that both gVisor and Firecracker run substantially more kernel code than native Linux and that neither is best on all workloads. Because gVisor is selectable as a Docker runtime (`-runtime=runsc`), it is the lowest-effort route to a stronger boundary for a Docker-based substrate (Section VII).

Lighter-weight process sandboxes such as nsjail, bubblewrap, and firejail apply the same namespace, seccomp, and cgroup primitives to a single process without an image or daemon. They are the natural comparison point for an operator who does not need Docker’s image ecosystem. Boxed uses Docker because agent workloads are specified as images (a Python runtime, a toolchain, a set of packages) and because the Engine API supplies the exec-attach stream the in-sandbox agent rides on; a process-sandbox backend behind the same interface is possible but is not evaluated here.

B. Lightweight virtual machines

Agache et al. [9] demonstrated that a stripped-down KVM VMM can boot a guest kernel in ~125 ms with

<5 MiB of memory overhead per VM, and Firecracker now powers AWS Lambda. Kata Containers [13] and Manco et al. [14] pursue the same hypervisor-as-isolation thesis from complementary angles. These systems optimize boot cost and steady-state memory under a serverless assumption: boot once, run an event loop, amortize startup across many invocations. An autonomous coding agent instead creates an ephemeral sandbox, runs one short program, extracts outputs, and destroys the environment, many times per task; the cumulative lifecycle cost, not cold start in isolation, decides whether a substrate is viable for that pattern.

C. Unikernels and WebAssembly

Madhavapeddy et al. [15] compile the application together with a library operating system into a single boot image, minimizing the trusted computing base at the cost of application-specific compilation, which rules out arbitrary dynamically generated binaries. WebAssembly [16] and WASI [17] make the guest portable bytecode with capability-based imports and the sandbox a runtime in the host process; microsecond instantiation is what a create-heavy agent workload wants, but agents emit unmodified Python scripts, shell commands, and native binaries that assume a POSIX environment, which the WASI toolchain does not yet run unmodified.

D. Agent-oriented sandbox platforms

A recent generation of products targets LLM agents specifically. E2B [6] runs Firecracker microVMs behind a SaaS API and reports ~ 150 ms cold start; Modal Sandbox [7] layers gVisor over Firecracker; Daytona [18] ships an AGPL Docker-backed self-hosted development environment; Cloudflare Sandbox [8] reuses V8 isolates for ~ 5 ms cold start at the cost of restricting execution to JavaScript and Wasm. On the research side, OpenHands [19] executes every agent action in a Docker container sandbox through a REST action execution API running inside it, the closest design sibling to Boxed’s driver plus agent split; it is an agent framework whose sandbox is one component, not a substrate with a backend interface. Cold-start figures for the commercial platforms are vendor claims measured under undisclosed conditions; Table I reproduces them as reported, not as validated, and they are not comparable to our end-to-end lifecycle measurement. Two limitations are shared across the category: the isolation mechanism is fixed at the platform level, and with the exception of Daytona and OpenHands, the platforms operate primarily as hosted services.

E. Evaluating LLM-generated code execution

Chen et al. [20] established single-function code generation as a standard benchmark. Yang et al. [1] extended this to agent-mediated software engineering, where the agent must run the code it writes and refine it. Our agent-trace evaluation (Section VI-F) follows the latter

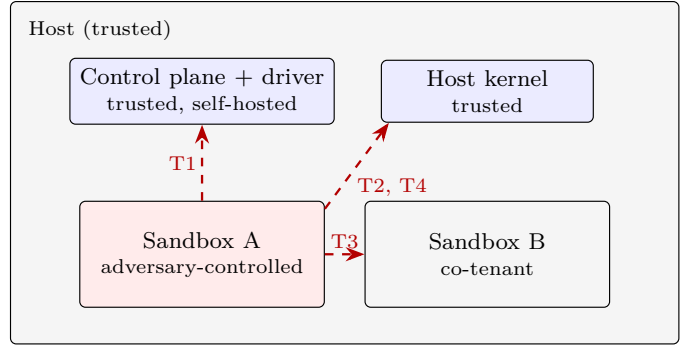


Fig. 1. Trust boundary. Adversary-controlled code executes inside a sandbox; the control plane, driver, and host kernel are trusted. Dashed arrows are the four threat classes the substrate must resist. Side channels between co-tenants on shared hardware are out of scope.

regime on official HumanEval tasks: the model emits a candidate Python function, Boxed executes it against the task’s published test, and the harness records the process exit status. Decomposing end-to-end wall-clock into model inference versus sandbox lifecycle quantifies the part of agent latency that a substrate can influence.

III. Threat Model

The threat model assumes an adversary with full control over the code submitted to a sandbox: they may craft arbitrary Linux binaries, issue any syscall sequence, and attempt network egress to any destination. The adversary does not control the control plane, the driver implementation, or the host kernel; these remain trusted. Fig. 1 shows the resulting trust boundary.

The design aims to prevent four classes of attack.

T1, host filesystem access. Read or write access to the host filesystem outside the sandbox’s ephemeral root, whether through path manipulation, symbolic links, or mount operations.

T2, namespace escalation. Escape from the allocated network or process namespace into the host’s network stack or process tree, or tampering with the in-sandbox agent from a workload process that shares its namespace.

T3, co-tenant exfiltration. Access to credentials, artifacts, or data belonging to a concurrent sandbox on the same host.

T4, resource exhaustion. Denial of service against the host through fork bombs, memory exhaustion, or other unbounded resource consumption.

Side-channel attacks exploiting shared hardware (cache timing, speculative execution) are out of scope. Mitigating them requires per-customer hardware partitioning, a deployment decision that belongs to the operator. The escape probe of Section VI-E covers T1, T2, and T4 directly; T3 is covered only through the network vectors, because the sandbox has no route to a co-tenant except the network, and no vector attempts a cross-sandbox read through the host. That gap is recorded in the limitations.

TABLE I

Agent-oriented sandbox runtimes (data as of 2026-05). Cold-start figures for systems other than Boxed are vendor claims measured under undisclosed conditions and are reproduced as reported, not validated by us; the Boxed figure is the end-to-end create+exec+destroy median of Section VI-B and includes an exec, which the vendor figures may not.

System	Isolation	Cold start	Backend selectable	Self-host	License	First-class artifacts
Boxed (this work)	Docker (runc)	178 ms (measured)	interface only*	yes	MIT	yes
E2B [6]	Firecracker	~150 ms (claim)	no	limited	Apache-2.0	yes
Modal Sandbox [7]	gVisor + Firecracker	~1.5 s (claim)	no	no	proprietary	yes
Daytona [18]	Docker	not reported	no	yes	AGPL	no
Cloudflare Sandbox [8]	V8 isolates	~5 ms (claim)	no	no	proprietary	limited

*The driver interface is designed for other backends; Docker is the only implementation in this release.

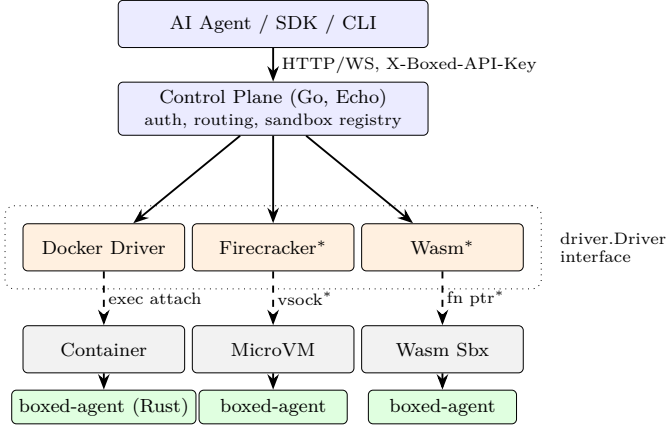


Fig. 2. Boxed architecture. The control plane drives one Driver interface. The Docker driver is the only implementation; the Firecracker and Wasm boxes show where a microVM or Wasm backend would attach. A common Rust agent runs inside every sandbox and speaks JSON-RPC over whatever byte stream the driver supplies. *Not implemented; shown as a design target only.

IV. Design

Boxed is structured around three components and one narrow interface (Fig. 2): a stateless control plane, a driver layer, and an in-sandbox agent. The design decouples orchestration from the isolation mechanism so that the same control plane could drive Docker containers on a development laptop and a microVM backend in production without modification. Only the Docker side of that sentence exists today.

A. Driver Interface

The central abstraction is a single Go interface. It has twelve methods in the current source; eight are file-injection, listing, health, and naming helpers used by the SDK and the control plane. The four lifecycle methods below are the minimal contract a backend must satisfy, and they are the only ones the control plane’s sandbox path depends on:

```
type Driver interface {
    Create(ctx context.Context,
        cfg SandboxConfig) (string, error)
    Start(ctx context.Context, id string) error
    Stop(ctx context.Context, id string) error
}
```

```
Connect(ctx context.Context,
    id string) (io.ReadWriteCloser, error)
// ... file, listing, and health helpers
}
```

Create instantiates a sandbox from the supplied configuration and returns an identifier; Start and Stop bound its lifetime; Connect returns a raw byte stream over which the control plane talks to the in-sandbox agent. For the Docker driver that stream is a Docker exec attach. A microVM driver would return a vsock connection and a Wasm driver a host-side function; the control plane does not know which case it is servicing. Whether four methods are in fact sufficient for a microVM backend is untested, because no such backend has been written, and we list it as future work rather than as a property of the design.

B. Control Plane

The control plane is a stateless HTTP and WebSocket server built on the Echo framework that maps POST /v1/sandbox requests through the configured driver. Two choices matter for agent workloads.

First, create returns once the underlying runtime reports that the sandbox process has launched (for the Docker driver, once ContainerStart acknowledges) with status: ready. The caller does not poll a separate status endpoint; the client sees one request-response exchange, and per-sandbox polling load disappears from the control plane. The first exec after create absorbs the cost of waiting for the agent’s JSON-RPC loop to come up. An agent task may create dozens of short-lived sandboxes, so removing a polling round-trip from each one is a material saving in aggregate.

Second, authentication uses an operator-configured static API-key header (X-Boxed-API-Key). Boxed does not issue, rotate, or store keys. This is a single-instance authentication mechanism, not a multi-tenant identity, key-management, or compliance system. In a self-hosted deployment, the operator controls the infrastructure that receives generated code, execution output, and intermediate files.

C. In-Sandbox Agent

Inside every sandbox runs boxed-agent, a single Rust binary built on tokio, serde, and notify (1.32 MiB stripped;

it links the C library dynamically and has no other runtime dependency). It speaks JSON-RPC 2.0 framed as newline-delimited JSON over the driver-supplied byte stream.

On exec, the agent spawns the user’s command as a child process and streams stdout and stderr chunks back as discrete RPC events as they appear. This matters for agent workloads in which the model must react to partial output or an early error signal before the program finishes. When the child exits, the agent reports the child’s exit status. An earlier release reported zero unconditionally; the correction is in the current source and its effect on the agent-trace results is discussed in Section VI-F. In parallel, an inotify-based watcher monitors the artifact directory (/output); any file written there is emitted as an artifact event carrying the file’s MIME type and base64-encoded contents. An agent that asks Python to write plot.png receives the bytes as an unsolicited event, with no second round-trip and no polling.

D. Artifact Protocol

Treating files as first-class outputs is what makes the substrate usable for workloads where the model expects the side effects of code, not only its console output.

The current release implements inline streaming: the watcher reads the file, guesses its MIME type, base64-encodes it, and emits it on the same channel that carries stdout and stderr. A size cap of 10 MiB bounds the memory and serialization cost of inlining; files above the cap are skipped with a logged warning. The wire protocol reserves a url field on the artifact event for an out-of-band upload path, in which the agent would PUT the file to a pre-signed URL and return only a reference, but that path is not implemented. We flag this because a substrate that silently drops large artifacts is a correctness hazard for any workload that emits model checkpoints or datasets.

Because both the artifact protocol and the agent live above the driver interface, artifact semantics do not depend on the backend.

V. Implementation

The release comprises 2,432 lines of Go in cmd and internal (control plane, Docker driver, CLI) and 918 lines of Rust for the in-sandbox agent. The control-plane binary is 12.0 MiB and the agent binary 1.32 MiB stripped. Three implementation details are stated precisely because two of them differ from what a reader might assume.

A. Agent injection and container lifecycle

The Docker driver bind-mounts the host’s boxed-agent binary into the container read-only at create time, so a stock base image (python:3.10-slim) can be reused across sandboxes with no Boxed-specific image registry to maintain. The image root filesystem is read-only. Separate tmpfs mounts back /tmp, /output, and the configured working directory, so user-written data is ephemeral and never touches the host filesystem.

The container’s own PID 1 is a tail -f /dev/null placeholder whose only job is to keep the namespace alive; boxed-agent is launched by Connect as a Docker exec whose attached stream carries JSON-RPC. The distinction has security consequences: the agent is a child of the container’s init, so a workload sharing the sandbox’s PID namespace can see it, and the ptrace vector of Section VI-E targets the agent for that reason. Under a microVM backend the agent would be the guest init, communicating over vsock.

B. Resource and network model

Every sandbox is created with a CPU quota (NanoCPUs), a memory cgroup, and a PidsLimit of 256. The container root is read-only, all Linux capabilities are dropped, and no-new-privileges is set. Docker’s default seccomp profile applies; Boxed does not yet ship a custom profile. The resource defaults are 1.0 core and 512 MiB, with ceilings of 4.0 cores and 8 GiB. A TTL goroutine stops the sandbox when its timeout expires.

The Docker backend always uses Docker’s none network, leaving only loopback. Requests that enable networking or specify an allow-list are rejected until Boxed implements an enforceable per-sandbox egress policy. This is a fail-closed restriction; it is not a claim that Docker’s default bridge is a sufficient egress control.

These settings are the ones the earlier pre-hardening release lacked. The May 2026 traces retained in the repository under bench/results/legacy-2026-05-docker-desktop were collected without the read-only root, the capability drop, the PID limit, and the network restriction, and with the exit-status defect in the agent; every number in Section VI comes from the hardened build.

VI. Evaluation

The evaluation answers the two questions posed in Section I: what the control plane and agent cost over calling Docker directly, and which escape vectors the hardened driver stops when judged by post-conditions. It also reports throughput under concurrency, idle per-sandbox overhead, and an end-to-end agent trace. The harness, the campaign runner, and every raw trace are released with the source; all numbers in this section expand from macros that bench/analyze/stats.py generates from those traces.

A. Setup

All measurements were taken on a MacBook Pro with an Apple M1 Pro SoC and 16 GB of unified memory running macOS 26.5, with Docker Engine 29.5 (linux/arm64, kernel 6.8, cgroup v2, seccomp and AppArmor enabled) inside a colima virtual machine using Apple’s Virtualization framework, provisioned with 4 vCPUs and 8 GiB. The python:3.10-slim image was pre-pulled. The machine ran ordinary background processes (editor, browser) throughout; it was not quiesced, and we report dispersion across repeated runs so that the effect of that contention is visible. Unless otherwise noted each lifecycle runs a synthetic

TABLE II

Lifecycle latency, medians over 5 runs $\times$ 200 sequential create $\rightarrow$ exec $\rightarrow$ destroy cycles, same host, image, and command. Raw Docker rows call the Engine API directly with no control plane and no agent.

Configuration	create	exec	destroy	total	Concurrency c	Sandboxes/s (mean)	95% CI	Range	
Raw Docker, stock defaults	90	53	73	216	499	1	6.98	± 0.42	5.70–7.59
Raw Docker, Boxed’s hardened config	76	46	29	153	379	2	11.44	± 0.97	9.16–13.49
Boxed (control plane + agent)	98	48	30	178	327	4	14.32	± 2.25	6.34–16.83
All values in ms.						8	16.43	± 2.98	9.99–21.29
						16	17.19	± 2.68	12.23–22.77
						32	17.14	± 2.41	11.73–20.94

print(‘ok’) workload, which isolates substrate overhead from application computation. The Boxed control plane was built from the tagged release with Go 1.24 and the agent with Rust 1.80.

B. Lifecycle latency against raw Docker

The substrate’s cost is only meaningful relative to what Docker itself costs on the same host. Three configurations were measured with an identical create $\rightarrow$ exec $\rightarrow$ destroy sequence: raw Docker, stock defaults, calling the Engine API directly with docker run’s default host configuration; raw Docker, hardened, calling the Engine API with exactly the host configuration Boxed’s driver applies (read-only root, CapDrop ALL, no-new-privileges, PID limit, network none, 1 CPU, 512 MiB, tmpfs mounts) but with no control plane and no agent; and Boxed, through the HTTP API. The raw configurations execute python3 -c "print(‘ok’)" through a Docker exec; Boxed executes the same command through the in-sandbox agent. Each configuration ran 5 runs of 200 sequential lifecycles, interleaved so that drift on the host affects all three equally.

Table II and Fig. 3 give the result. Boxed’s median end-to-end lifecycle is 178 ms (bootstrap 95% CI 177–179 ms; per-run medians 172 ms to 190 ms), with an interquartile range of 33 ms and a tail of p95 327 ms and p99 578 ms. Raw Docker with the same hardened configuration takes 153 ms. The difference, 25 ms (95% CI on the difference of medians 23–27 ms; 16% of the raw cost), is what the HTTP control plane, the agent launch over exec attach, and the JSON-RPC exec path add together. By phase, Boxed adds 22 ms to create, 2 ms to the first exec, and 1 ms to destroy. That is the price of an HTTP API any language can call, an agent that streams output and files as events, and a one-round-trip create; an operator who already drives the Engine API from Go and does not need streaming artifacts would not pay it.

The second comparison is between the two raw-Docker rows. Applying Boxed’s hardening to a stock docker run is 63 ms faster end to end. The stock configuration attaches the container to the default bridge network, and creating and tearing down that attachment costs more than the tmpfs mounts and capability drop cost; with network none that work disappears. The hardening that roll-your-own deployments omit is therefore not a latency trade-off on this host.

TABLE III

Throughput sweep: 80 create $\rightarrow$ destroy lifecycles per level, 10 independent sweeps. Mean sandboxes/s with 95% confidence interval (Student’s t) and the range across sweeps. 0 errors in 4800 lifecycles.

Concurrency c	Sandboxes/s (mean)	95% CI	Range
1	6.98	± 0.42	5.70–7.59
2	11.44	± 0.97	9.16–13.49
4	14.32	± 2.25	6.34–16.83
8	16.43	± 2.98	9.99–21.29
16	17.19	± 2.68	12.23–22.77
32	17.14	± 2.41	11.73–20.94

C. Throughput under concurrency

Client concurrency was swept over $c \in \{1, 2, 4, 8, 16, 32\}$ with 80 create $\rightarrow$ destroy lifecycles per level, and the whole sweep was repeated 10 times (Table III, Fig. 4). Throughput rises from 7.0 sandboxes/s at $c=1$ to a plateau of 17.2 (± 2.7) at $c=16$ and 17.1 at $c=32$; the last three levels are within one another’s confidence intervals. The VM has four vCPUs and one Docker daemon serializes every container operation inside it, which is where the plateau comes from. The range column shows why repeated sweeps matter: a single sweep at $c=4$ or $c=8$ can land anywhere in a factor-of-two band, and the earlier version of this work drew a shape (a dip at $c=16$, a collapse at $c=32$) from one sweep per level that these repeats do not reproduce.

D. Idle per-sandbox overhead

Each of 30 idle sandboxes was sampled five seconds after create returned. Median working-set size, as reported by docker stats from the container cgroup, is 0.43 MiB (p95 0.67 MiB, maximum 0.68 MiB); median CPU is 0.01%. The figure reflects cgroup accounting inside the VM, not a process-level measurement on a native host, so we read it as an order of magnitude. That order of magnitude is consistent with the design: the agent is a stripped Rust binary that holds no buffers and parks on one tokio event loop awaiting a JSON-RPC frame, where a Python- or Node-based agent would carry an interpreter and module cache even when idle.

E. Escape probe with post-condition checks

The hardened driver was exercised with twelve attempts drawn from common container-escape patterns: mounting a filesystem, reading the host through /proc/1/root, reaching the Docker or containerd socket, loading a kernel module, opening a raw socket, creating a new user and PID namespace, egress to an RFC1918 address, egress to the link-local metadata endpoint, a fork storm, a memory bomb that touches every page it allocates, a write to the root filesystem, and PTRACE_ATTACH on the in-sandbox agent. Each attempt runs in a fresh sandbox and the whole suite was run 3 times.

Each vector receives two verdicts on the same attempt. The signature verdict is the method used in the earlier

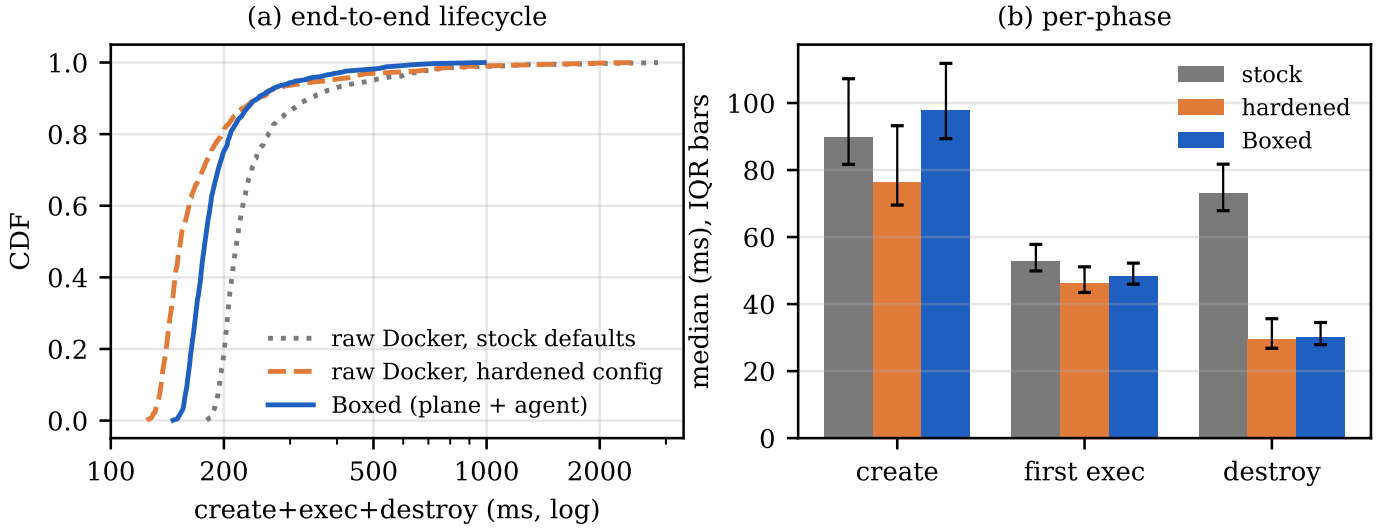


Fig. 3. Lifecycle latency, 5 runs $\times$ 200 lifecycles per configuration. (a) CDF of end-to-end create+exec+destroy. (b) Median per phase with interquartile bars. The difference between the Boxed and hardened raw-Docker bars is the cost of the control plane and the in-sandbox agent.

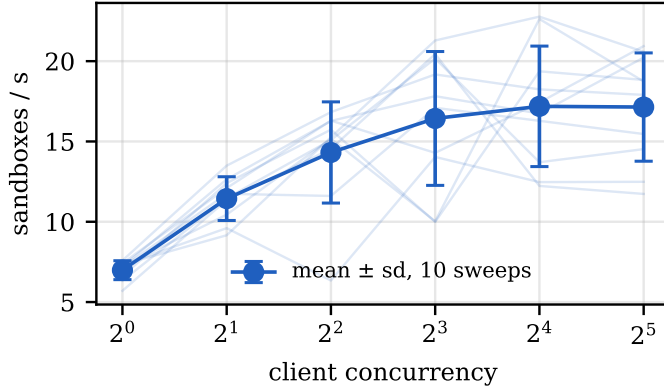


Fig. 4. Sustained sandbox-creation rate against client concurrency. Thin lines are the 10 individual sweeps; markers are the mean with one standard deviation.

version of this work: capture stdout and stderr and grep for a denial string. The post-condition verdict checks state after the attempt: whether a mount appeared in `/proc/mounts`, whether the written file exists, the errno the syscall itself returned, the device and inode that `/proc/1/root` resolves to, and, for the resource vectors, the cgroup’s `pids.events` and `memory.events` counters read on the host outside the sandbox. Table IV gives both verdicts and the evidence for every vector.

The post-condition checks classify 12 of 12 vectors as denied (100%), with 0 undenied and 0 harness errors, and the verdicts were identical across the 3 runs. Four observations follow from the evidence column.

First, the vectors that the earlier signature method scored as artifacts resolve cleanly. The memory bomb is killed by the cgroup, which the host’s `oom_kill` counter records, while the process itself emits nothing before

SIGKILL; the signature method reads that silence as a success for the adversary. The fork storm is granted exactly the forks the PID limit allows and then receives EAGAIN, and the host’s `pids.events` counter shows the limit binding. `/proc/1/root` resolves to the same device and inode as the container’s own root, so listing it crosses no boundary. The Docker socket check finds no socket, which is the denial, not an artifact of it.

Second, the network denials are now Boxed policy. In the pre-hardening configuration the sandbox sat on Docker’s bridge and the RFC1918 and metadata denials were plausibly Docker Desktop’s NAT topology; with network none the sandbox has only a loopback interface, which the evidence column shows, and no route exists to deny.

Third, the ptrace vector, which the pre-hardening driver did not stop, is denied on the hardened driver because CapDrop ALL removes `CAP_SYS_PTRACE` and the kernel’s Yama scope of 1 restricts attachment to descendants; the agent is a sibling of the workload, not its child. This is a property of the capability drop and the host kernel’s Yama setting together, and a host with Yama disabled would need the seccomp profile Boxed does not yet ship. The module-load vector was likewise scored by calling `init_module` directly; the earlier probe invoked a `modprobe` binary the image does not contain, so its “not found” verdict measured the image, not the boundary.

Fourth, the signature and post-condition verdicts disagree on 2 of 12 vectors on the same run. Every disagreement is the signature method reporting an undenied attempt that the post-condition evidence shows was stopped. A probe scored by strings in captured output therefore understates a sandbox’s coverage on exactly the vectors where the enforcement mechanism kills or silences

TABLE IV

Escape probe on the hardened Docker driver, 12 vectors, 3 independent runs (verdicts identical across runs). Signature is the v1 classifier (grep for a denial string in captured output); Post-condition checks state after the attempt, on the host where possible. The paper reports the post-condition column.

Vector	Threat	Exit	Signature	Post-condition	Evidence
mount_host	T1, T2	0	denied	denied	mounts_on_/mnt=0
proc1_root	T1	0	denied	denied	proc1_root_dev_ino=42:1193209 root_dev_ino=42:1193209 pid1=tail
docker_sock	T2, T3	0	undenied	denied	sockets_found=0 api_reachable=False
modprobe	T2	0	denied	denied	init_module errno=1(Operation not permitted)
raw_socket	T2	0	denied	denied	errno=1(Operation not permitted)
setns	T2	0	denied	denied	unshare rc=1 unshare: unshare failed: Operation not permitted
rfc1918	T3	0	denied	denied	ifaces=lo
imds	T3	0	denied	denied	ifaces=lo
fork_bomb	T4	0	denied	denied	forked=250 EAGAIN after 250 forks host:pids.max=256 pids.events=max 1
mem_bomb	T4	-1	undenied	denied	exit=-1 host:memory.max=536870912 oom_kill 1
ro_root	T1	0	denied	denied	/etc:absent /usr/bin:absent
ptrace_agent	T2, T3	0	denied	denied	PTRACE_ATTACH pid=7 errno=1(Operation not permitted) yama=1

the process, which is the mechanism for the resource class. We report this because the earlier version of this work published the signature verdicts, and the correction is part of the record.

This is a behavioural probe with twelve vectors, not an isolation proof. It does not exercise kernel vulnerabilities, it runs one image, and it does not include a cross-sandbox read (T3). None of the twelve attempts produced a host escape.

F. End-to-end agent trace

Boxed was driven by a code-generation loop over the first 20 tasks of the official HumanEval release [20] in file order (HumanEval/0 onward), repeated 3 times for 60 task executions. For each task the model (claude-opus-5, default adaptive thinking, 4096 max output tokens, called through the Anthropic Messages API with no fallback model so that every completion is attributable to the named model) receives the task’s signature and docstring and returns the function; Boxed executes the prompt, the completion, and the task’s published check function in a fresh sandbox, and the process exit status decides pass or fail. A deliberately wrong candidate sent through the same path exits 1 with an AssertionError, which is the control the earlier release would have failed. The model identifier, timestamp, prompt hash, token counts, stop reason, raw completion, executed program, and captured output for every task are in agent_trace.jsonl beside each CSV.

60 of 60 task executions passed (100%; stop reasons end_turn: 60). Median end-to-end wall-clock was 3.95s, of which the model call took 3.42s and the sandbox lifecycle 453 ms (per-repetition medians 436–504 ms; Fig. 5). Summed over all executions, model inference is 85% of wall-clock (80–88% across repetitions) and the sandbox 15%. Sandbox create in this loop had a median of 279 ms (268–328 ms across repetitions) against 98 ms in the tight lifecycle loop of Section VI-B. The host’s one-minute load average was between 13 and 17 on eight cores during all three repetitions, from activity unrelated to

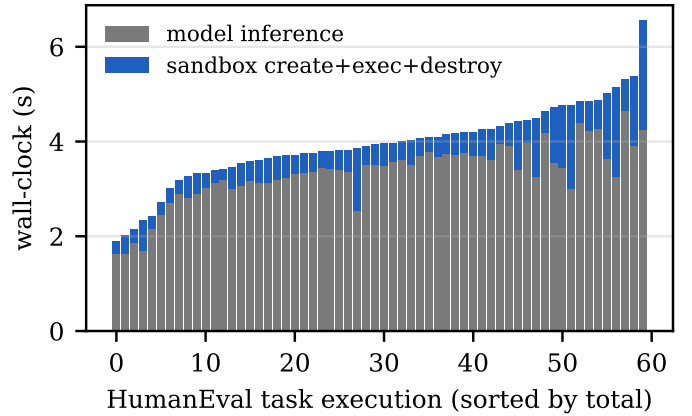


Fig. 5. Model inference versus sandbox time per task execution over 60 HumanEval task executions (3 repetitions of 20 tasks); a red mark would denote a failed task.

the experiment, where the lifecycle campaign ran at lower load; the recorded load is kept beside the traces. The gap is therefore an upper bound on what an idle gap between creates costs, and on a quieter host the model’s share would be larger still. The pass rate is a property of the model on HumanEval, not of Boxed; what the trace establishes for the substrate is that the exit status carries the test outcome, and what share of an agent iteration the sandbox occupies with a current frontier model.

G. Threats to validity

All measurements come from one laptop, and Docker ran inside a virtual machine, so absolute latencies include a hypervisor and virtio path that a native Linux host would not pay; the earlier campaign on Docker Desktop’s LinuxKit VM measured 303 ms for the same lifecycle on the unhardened driver, and the two VM stacks are not comparable. The comparison that the paper rests on, Boxed against raw Docker, holds the host, VM, image, and command fixed, and the interleaved runs give a dispersion estimate; that comparison is robust to the platform in a

TABLE V
Hardening checklist for the Docker driver.

Pre-hardening gap	Vector	Status in this release
Writable rootfs	ro_root	fixed: ReadonlyRootfs; confirmed
Default capability set	raw_socket, modprobe	fixed: CapDrop ALL; confirmed
No process cap	fork_bomb	fixed: PidsLimit 256; confirmed
Permissive ptrace	ptrace_agent	denied via capability drop and Yama; custom seccomp pending
Bridge network by default	rfc1918, imds	fail closed: network none; egress policy pending
Oversize artifacts dropped	(none)	upload path pending

way the absolute numbers are not. The host was not quiesced. The escape probe is twelve vectors on one image and tests configuration, not kernel vulnerabilities. Throughput lifecycles are create→destroy without an exec, so they measure the daemon’s container churn rate, not agent-visible latency. The comparison figures in Table I for other systems are vendor claims we did not reproduce. The agent trace uses one model and twenty tasks, three times, under a loaded host.

VII. Discussion

A. Hardening status

Table V maps each gap the pre-hardening probe exposed to its status in the current source. The read-only root, capability drop, PID limit, privilege-escalation block, and network restriction are in place and the post-condition probe confirms each of them. Two items remain: a custom seccomp profile, so that the ptrace denial does not depend on the host’s Yama setting, and a per-sandbox egress allow-list, until which networked execution is refused. The upload path for artifacts above the inline cap is also unimplemented.

B. gVisor as the next boundary

Because Docker accepts runsc as a container runtime, the cheapest stronger boundary for this substrate is a one-field change to the driver’s host configuration, with the same image, agent, and control plane. It was not measured here because gVisor is not supported inside the macOS virtual machine used for the campaign.

C. Pool-based sandbox reuse

The driver creates a fresh container per request. That buys a known-clean starting state at the cost of paying 98ms of container instantiation on every call, which Table II shows is the dominant phase for Boxed and for raw Docker alike. Reusing a warmed sandbox for compatible follow-up requests would amortize that cost, as Oakes et al.’s SOCK [21] does for serverless functions with an LRU cache of paused containers. Compatibility is naturally keyed on base image, network policy, and resource limits; a reset between uses (killing descendant

processes, clearing /tmp and /output) is far cheaper than a teardown and recreate, and the interface already allows Create to return an existing identifier. For a workload that spawns dozens of sandboxes per task this is the highest-leverage optimization remaining, and its cost in isolation strength should be measured, not assumed.

VIII. Future Work

Four directions follow from the limitations reported above. A second backend. The interface is designed so a stronger isolation backend can be substituted without altering the control plane, and that claim is untested until a second backend exists; gVisor through runsc on a native Linux host is the cheapest test, and a Firecracker driver the intended one. A custom seccomp profile and egress policy, so that the ptrace denial and the network restriction are Boxed’s own rather than the host’s. Sandbox reuse, measured for both the latency it saves and the isolation it weakens. Native Linux measurement, to separate the cost of the virtual machine from the cost of the substrate. Beyond these, a multi-host scheduler once aggregate concurrency exceeds one machine, and a Wasm/WASI backend once the toolchain runs unmodified language runtimes (Section II).

IX. Conclusion

Boxed is a Docker-based execution substrate for LLM agents that create, exercise, and destroy sandboxed environments many times per task. It provides a streaming in-sandbox agent, a one-round-trip create, and a self-hosted control plane, behind an interface intended to admit stronger backends. On the same host, image, and command, the substrate costs 25ms over calling Docker directly, and the hardened configuration it applies is 63ms faster than stock Docker defaults. Judged by post-conditions read from the host, the hardened driver stops 12 of 12 escape vectors, and the signature-matching method used in the earlier version of this work would have reported 2 of those as failures. What remains open is stated in Section VII: a custom seccomp profile, an egress policy, an artifact upload path, and a second backend that would turn the interface’s design intent into a measured property. Implementation, harness, campaign runner, and all raw traces are MIT-licensed at <https://github.com/akshayaggarwal99/boxed> (tag v0.2.0-paper); bench/analyze/stats.py regenerates every number, table, and figure in this paper from them.

Conflict of Interest

The author maintains Boxed as an open-source side project outside of employment and has no business relationship with, and receives no funding from, Docker, Anthropic, or any vendor in Table I. The raw-Docker baseline, the interleaved runs, and the released harness mitigate the evaluator also being the author.

References

- [1] J. Yang, C. E. Jimenez, A. Wettig et al., “SWE-agent: Agent-computer interfaces enable automated software engineering,” in NeurIPS, 2024.
- [2] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable code actions elicit better LLM agents,” in International Conference on Machine Learning (ICML), 2024.
- [3] NIST NVD, “CVE-2019-5736: runc container breakout,” <https://nvd.nist.gov/vuln/detail/CVE-2019-5736>, 2019.
- [4] X. Lin, L. Lei, Y. Wang, J. Jing, K. Sun, and Q. Zhou, “A measurement study on Linux container security: Attacks and countermeasures,” in Annual Computer Security Applications Conference (ACSAC), 2018.
- [5] X. Gao, Z. Gu, Z. Li, H. Jamjoom, and C. Wang, “Houdini’s escape: Breaking the resource rein of Linux control groups,” in ACM SIGSAC Conference on Computer and Communications Security (CCS), 2019.
- [6] E2B, “E2B: Open-source secure sandboxes for AI agents,” <https://e2b.dev>, 2024.
- [7] Modal Labs, “Modal Sandbox documentation,” <https://modal.com/docs/guide/sandbox>, 2025.
- [8] Cloudflare, “Cloudflare Sandbox SDK,” <https://developers.cloudflare.com/workers/sandbox/>, 2025.
- [9] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, “Firecracker: Lightweight virtualization for serverless applications,” in USENIX NSDI, 2020.
- [10] Google, “gVisor: Container runtime sandbox,” <https://gvisor.dev>, 2018.
- [11] E. G. Young, P. Zhu, T. Caraza-Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “The true cost of containing: A gVisor case study,” in USENIX Workshop on Hot Topics in Cloud Computing (HotCloud). USENIX Association, 2019.
- [12] Anjali, T. Caraza-Harter, and M. M. Swift, “Blending containers and virtual machines: A study of Firecracker and gVisor,” in ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE), 2020, pp. 101–113.
- [13] Kata Containers Project, “Kata Containers: Secure containers with the speed of containers,” Open Infrastructure Foundation project, <https://katacontainers.io>, 2018.
- [14] F. Manco, C. Lupu, F. Schmidt, J. Mendes, S. Kuenzer, S. Sati, K. Yasukata, C. Raiciu, and F. Huici, “My VM is lighter (and safer) than your container,” in SOSP, 2017.
- [15] A. Madhavapeddy, R. Mortier, C. Rotsos, D. Scott et al., “Unikernels: Library operating systems for the cloud,” in ASP-LOS, 2013.
- [16] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. Bastien, “Bringing the web up to speed with WebAssembly,” in ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2017.
- [17] Bytecode Alliance, “WebAssembly system interface (WASI),” <https://wasi.dev>, 2024.
- [18] Daytona, “Daytona: Open-source dev-environment platform,” <https://daytona.io>, 2024.
- [19] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hou, Y. Yang, L. Shirodkar, M. Schmidt, R. Zhang, A. Nguyen, and G. Neubig, “OpenHands: An open platform for AI software developers as generalist agents,” in International Conference on Learning Representations (ICLR), 2025, arXiv:2407.16741.
- [20] M. Chen, J. Tworek, H. Jun et al., “Evaluating large language models trained on code,” arXiv preprint arXiv:2107.03374, 2021.
- [21] E. Oakes, L. Yang, D. Zhou, K. Houck, T. Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “SOCK: Rapid task provisioning with Serverless-Optimized containers,” in USENIX Annual Technical Conference (USENIX ATC). USENIX Association, 2018, pp. 57–70.